

Request and Response in PHP

Following `request_info.php` from top to bottom

Learning Goals

- Run the actual `request_info.php` example
- Read request information from `$_SERVER`
- Read query parameters from `$_GET`
- Extract a path with `parse_url()`
- Return JSON with the correct response header

IMPORTANT

Start the Example

From this lesson's code directory, start PHP:

```
php -S localhost:8000
```

In another terminal, send a GET request:

```
curl "http://localhost:8000/request_info.php?name=Alice"
```

The **Request** PHP Receives

```
GET /request_info.php?name=Alice HTTP/1.1  
Host: localhost:8000  
User-Agent: curl/8.x
```

The request contains a method, path, headers, and query string.

- PHP parses these pieces before **request_info.php** runs.

Where the Request Goes

- PHP runs only the file named in the path:
`/request_info.php`.
- We use `curl` because the terminal shows the request and response directly.
- In a browser, enter the same URL in the address bar; the browser displays the JSON response.

IMPORTANT

PHP Superglobals in This Example

```
$_SERVER['REQUEST_METHOD']  
$_SERVER['REQUEST_URI']  
$_SERVER['HTTP_USER_AGENT']  
$_GET
```

- `$_SERVER` contains request and server information.
- `$_GET` contains decoded query parameters.

IMPORTANT

Open `request_info.php`

```
<?php  
declare(strict_types=1);  
  
header('Content-Type: application/json');
```

`Content-Type` tells the client what format the response body uses.

- PHP sends `Content-Type: application/json` before the response body.
- The client reads this header, then treats the body as JSON

Build the Response Array

```
$response = [  
    'method' => $_SERVER['REQUEST_METHOD'],  
    'path' => parse_url($_SERVER['REQUEST_URI'], PHP_URL_PATH),  
    'query' => $_GET,  
    'user_agent' => $_SERVER['HTTP_USER_AGENT'] ?? null,  
];
```

Each response field comes directly from the current request.

Read the Method

```
curl "http://localhost:8000/request_info.php?name=Alice"
```

```
'method' => $_SERVER['REQUEST_METHOD'],
```

For this `curl` command, the value is `GET`. PHP provides the method directly;

- the application does not split the raw request line.
- `curl` sends a `GET` request by default when given only a URL.

IMPORTANT

GET Requests

- **GET** asks the server to read or retrieve data.
 - Its query parameters are usually included in the URL.
 - In `?name=Alice`, `name` is a condition for what to read; a **GET** request should not save or change server data.
- Later, we will use **PUT** to update existing data.

Parsing Request Information

PHP parses the raw request and makes its information easy to retrieve.

- Earlier, `$_SERVER['REQUEST_METHOD']` gave us the method.
- The same `$_SERVER` array provides more request details.
- `$_GET` provides parsed query parameters.

IMPORTANT

Extract the Path

```
'path' => parse_url($_SERVER['REQUEST_URI'], PHP_URL_PATH),
```

`$_SERVER['REQUEST_URI']` contains the requested path and query string.

- `$_SERVER['REQUEST_URI']`: `/request_info.php?name=Alice`
- `PHP_URL_PATH` tells `parse_url()` to return only the path: `/request_info.php`
- The query string, `?name=Alice`, is handled separately by `$_GET`.

| Read Query Parameters with `$_GET`

PHP puts request information into superglobals:

- `$_SERVER`: method, URI, and headers
- `$_GET`: parsed query parameters

When building `$response`, add the query parameters as one field:

```
$response = [  
    'query' => $_GET,  
];
```

From Query String to `$_GET`

```
curl "http://localhost:8000/request_info.php?name=Alice&course=ASE230"
```

The part after `?` is the query string. Before the script runs, PHP turns it into `$_GET`:

- splits parameters at `&`
- separates each `key=value` pair

For `?name=Alice&course=ASE230`, PHP provides:

```
$_GET = ['name' => 'Alice', 'course' => 'ASE230'];
```

Read one value with `$_GET['name']`.

- PHP URL-decodes values before adding them to `$_GET`.

URL Encoding and Decoding

URLs cannot contain spaces directly, so the client encodes them:

- `Alice Smith` becomes `Alice%20Smith`
- In form-style query strings, `Alice+Smith` also means `Alice Smith`

When PHP reads the query string, it decodes these values:

```
$_GET['name'] // "Alice Smith"
```

Handle an Optional Header

`User-Agent` is an HTTP request header that identifies the client software.

- For example, `curl` sends a value such as `curl/8.x`.
- The server can read it to see whether a browser, `curl`, or another tool sent the request.

```
'user_agent' => $_SERVER['HTTP_USER_AGENT'] ?? null,
```

The null-coalescing operator uses `null` when the client did not send a `User-Agent` header.

- This avoids an undefined-array-key warning.

IMPORTANT

JSON for Data Exchange

HTTP can send many formats.

- Web pages for people commonly use HTML.
- APIs commonly use JSON to send structured data between a client and server.
- JSON is a key format for exchanging data over HTTP.

Encode and Send JSON

```
echo json_encode(  
    $response,  
    JSON_PRETTY_PRINT | JSON_UNESCAPED_SLASHES  
);  
echo PHP_EOL;
```

- `JSON_PRETTY_PRINT` makes the output readable.
- `echo PHP_EOL;` starts a new line after the JSON, which keeps terminal output tidy when using `curl`.

Without it:

```
{"method": "GET", "path": "/request_info.php"}
```

With it:

```
{  
    "method": "GET",  
    "path": "/request_info.php"  
}
```

- `JSON_UNESCAPED_SLASHES` keeps `/request_info.php` easy to read.

For example, `['path' => '/request_info.php']` becomes:

```
{  
  "path": "/request_info.php"  
}
```

Expected Response

```
{  
  "method": "GET",  
  "path": "/request_info.php",  
  "query": {"name": "Alice"},  
  "user_agent": "curl/8.x"  
}
```

The exact `user_agent` version may differ on your computer.

Try Another Request

```
curl "http://localhost:8000/request_info.php?name=Alice%20Smith&year=2"
```

Predict `method`, `path`, `query.name`, and `query.year`, then run it.

Key Takeaways

1. `$_SERVER` provides request metadata.
2. `$_GET` contains query parameters.
3. `parse_url()` separates the path from the query string.
4. `header()` sets response metadata.
5. `json_encode()` creates the response body.